0 5 2

T fixtures — `io_trace.sh` RED-test corpus (plan.md T-C)

Revision: 1 **Last modified:** 2026-09-28T23:55:00+05:00

Contract: `specs/004-fast-dev-cycles/plan.md` §T-C02 ("Observed-I/O capture and the gate→inputs map") + FR-005/FR-007 in `specs/004-fast-dev-cycles/spec.md`. Protecting-tests line, verbatim from plan.md: "control needle: a gate known to read `CLAUDE.md` must list it; golden-bad: a gate that reads through a dynamically built path must still record the resolved path; paired mutation: filter out `stat` calls → a gate that decides on file existence loses an input and the needle FAILs." tasks.md T052 adds a third, explicitly-named negative control (the CT-5 addition — plan.md line 128): "a gate reading only its own script directory yields exactly that read set and PASSes".

The tool under test (not yet implemented — T-C , a later task)

`constitution/scripts/fastcycle/gates/io_trace.sh`, per plan.md's path table (line 186: `io_trace.sh # observed-I/O capture + gate→inputs map build (T-C02)`). Per T-C02's own "Work" line, the real implementation runs each gate under `strace -f -e trace=openat,stat,execve` in a clean checkout and records every path read and every path written, building a gate→reads/ gate→writes map.

Assumed CLI contract (UNCONFIRMED by the contract itself — no contracts/ file names this tool explicitly; DEFINED here for this RED test's own GREEN-branch, binding-if-adopted on T-C02's implementer, following the house precedent in `fixtures/verdict_cache/README.md` and `fixtures/dispatch_stamp/`):

```
io_trace.sh <gate-script> [gate-args...]
```

Runs `<gate-script> [gate-args...]` under strace, and on stdout emits ONE JSON object:

```
{"reads": ["<absolute path>", ...], "writes": ["<absolute path>", ...]}
```

Exit 0 on a successful trace (regardless of the traced gate's own exit code — the tracer's[3] job is to observe, not to judge the gate); nonzero only on a tracer-level failure (gate script missing, strace unavailable, etc).

The `scenarios`

Directory	Scenario	Protecting-tests line it satisfies
`needle_gate.sh` (no subdir — single-file fixture)	**Control needle**	"a gate known to read `CLAUDE.md` must list it"
`dynamic_path_gate/`	**Golden-bad**	"a gate that reads through a dynamically built path must still record the resolved path"
`own_dir_only_gate/`	**Negative control (CT-5)**	"a gate reading only its own script directory yields exactly that read set and PASSes"

`needle_gate.sh` — control needle

Reads the REAL project `CLAUDE.md` (never a copy — the needle must be a genuinely known-present value per §11.4.273, and this repo's own top-level `CLAUDE.md` is that value in any checkout of it). `io_trace.sh needle_gate.sh <repo-root>` must list a path ending in `CLAUDE.md` in its `reads` array.

`dynamic_path_gate/` — golden-bad

The gate script builds its target file's path from three concatenated shell variables (`base` + `_data` + `ext`) rather than a single literal, so a naive static-analysis (grep-the-source) approach would see only the unexpanded template string, never the real path. `io_trace.sh` MUST record the RESOLVED absolute path ending in `target_data.txt` — this is why T-C02 mandates strace-based OBSERVED tracing (the syscall layer only ever sees post-expansion paths) rather than[5] source scanning.

`own_dir_only_gate/` — negative control (CT-)

The gate reads ONLY files inside its own directory: itself (the shell interpreter opens the script file to execute it) and one sibling data file. `io_trace.sh` 's reported read-set for this gate MUST equal EXACTLY `{own_dir_only_gate.sh, sibling_data.txt}` — no extra basenames (the real implementation must filter dynamic-linker/shared-library noise strace also observes) and no omissions. This is the dual false-positive/false-negative guard (§11.4.201(1)) for the tracer mechanism itself, not merely a "does it find something" smoke test.

What this RED test does NOT cover

- The `writes` side of the map (no fixture gate here performs a write) — T-C02's own implementer is expected to add write-side fixtures alongside the reads-side ones landed here, or a follow-up task does.
- The paired mutation named in plan.md ("filter out `stat` calls → a gate that decides on file existence loses an input and the needle FAILs") — that is a mutation of the REAL implementation once it exists (T-C02), not something a RED test against an absent tool can exercise; T179's batched mutation sweep is where it belongs.
- Re-tracing on gate-script-hash-change and the "fully on the backstop" cadence rule from T-C02's Work line — those are implementation-internal caching/ scheduling behaviour, not I/O-observation-correctness fixtures.